

1. Longest Repeating Character Replacement → Window + Frequency Map

```
s = "AABABBA"
k = 1
```

```
def char_rep(s,k):

    count = {}
    left = 0
    max_freq = 0
    max_len = 0

    for right in range(len(s)):

        char = s[right]

        count[char] = count.get(char, 0) + 1

        max_freq = max(max_freq, count[char])

        while (right - left + 1) - max_freq > k:
            count[s[left]] -= 1
            left += 1

        win_len = right - left + 1

        max_len = max(max_len, win_len)

    return max_len

print(char_rep(s,k))
```

Optimized Version:

```
def check_inclusion(s1, s2):

    if len(s1) > len(s2):
        return False

    target = [0] * 26
    window = [0] * 26

    for char in s1:
        target[ord(char) - ord('a')] += 1

    left = 0

    for right in range(len(s2)):

        window[ord(s2[right]) - ord('a')] += 1

        if right - left + 1 > len(s1):

            window[ord(s2[left]) - ord('a')] -= 1
            left += 1

        if window == target:
            return True

    return False

print(check_inclusion(s1,s2))
```

2. Permutation in String → Fixed Window + Frequency

Dictionary Version:

```
s1 = "ab"
s2 = "eidbaooo"
```

```
def check_inclusion(s1, s2):

    if len(s1) > len(s2):
        return False

    target = {}

    for char in s1:
        target[char] = target.get(char, 0) + 1

    window = {}

    left = 0
    k = len(s1)

    for right in range(len(s2)):

        char = s2[right]

        window[char] = window.get(char, 0) + 1

        if right - left + 1 == k:

            if window == target:
                return True

            left_char = s2[left]

            window[left_char] -= 1

            if window[left_char] == 0:
                del window[left_char]

            left += 1

    return False

print(check_inclusion(s1,s2))
```

3. Minimum Window Substring → Variable Window + HashMap

```
s = "ADOBECODEBANC"
t = "ABC"

def min_window(s, t):

    if not s or not t:
        return ""

    need = {}

    for char in t:
        need[char] = need.get(char, 0) + 1

    window = {}

    required = len(need)
    formed = 0

    left = 0

    min_length = float('inf')
    min_left = 0

    for right in range(len(s)):

        char = s[right]

        window[char] = window.get(char, 0) + 1

        if char in need and window[char] == need[char]:
            formed += 1

        while formed == required:

            current_length = right - left + 1

            if current_length < min_length:

                min_length = current_length
                min_left = left

            left_char = s[left]

            window[left_char] -= 1

            if (
                left_char in need
                and window[left_char] < need[left_char]
            ):
                formed -= 1

            left += 1

        if min_length == float('inf'):
            return ""

    return s[min_left:min_left + min_length]

print(min_window(s,t))
```